PROJECT REPORT

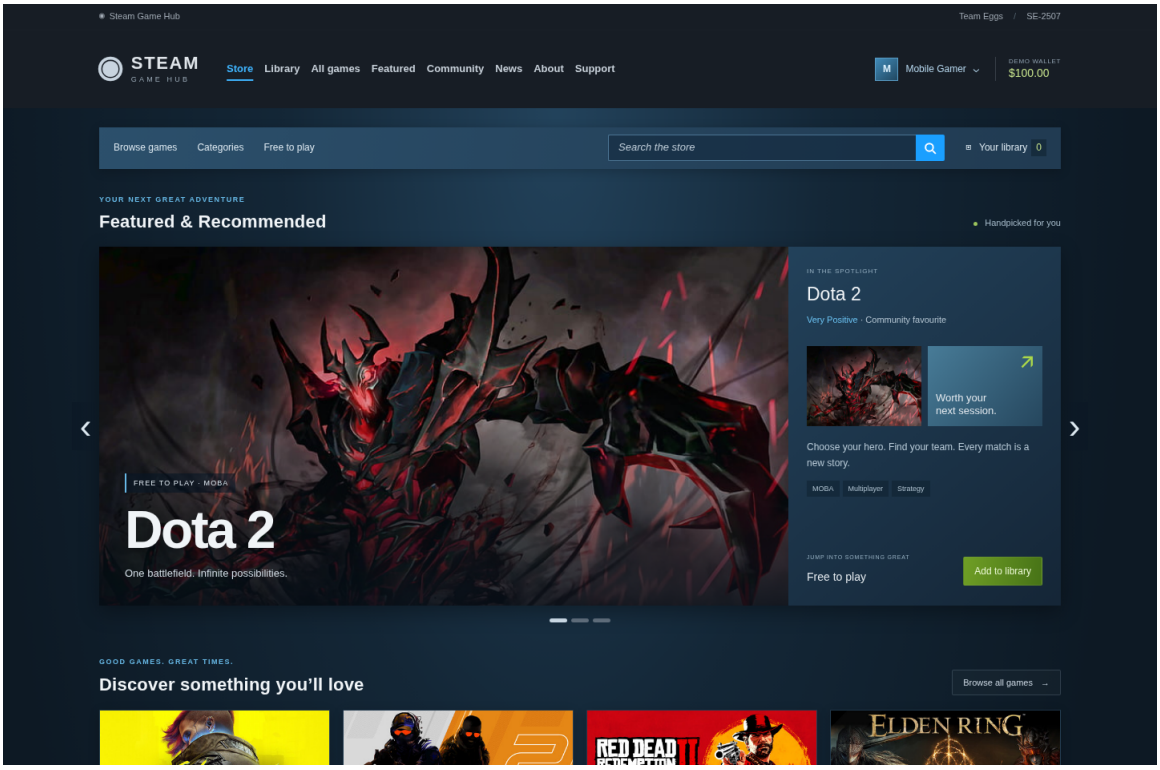
Assignment 6. jQuery

A game storefront with live discovery, player ratings, virtual purchases and returns.

Objective. Use jQuery for DOM changes, events, animations and practical frontend interactions while continuing our existing team website.

Team: Eggs
Group: SE-2507
Members: Ташкенбаев Нурсултан; Бигали Абулхаир; Султанкулов Шынгыс; Лопаткин Артем.

Stack: HTML, CSS, Bootstrap 5.3.8, jQuery 4.0.0 and browser localStorage. No backend, real accounts, payment provider or server requests for purchases.



The redesigned desktop storefront. Full-resolution screenshots accompany this report.

Website: https://abilkhayrbigali-design.github.io/
Repository: github.com/abilkhayrbigali-design/abilkhayrbigali-design.github.io

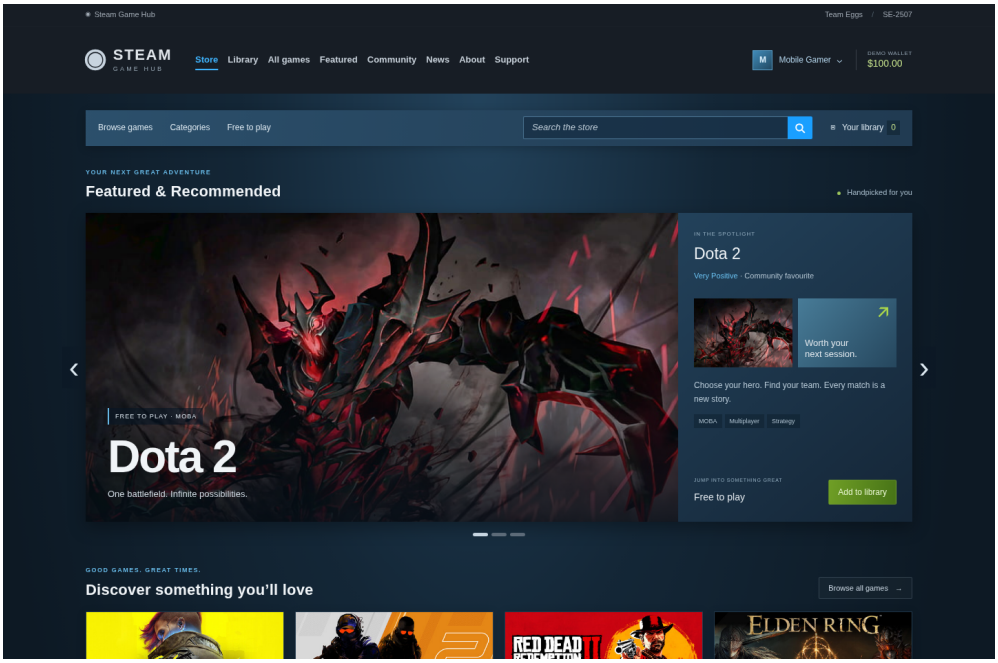
Verification: 57/57 browser checks passed in Chromium 154.0.8037.97, including 10 pages at 375, 768 and 1440 px.

TASK 0

Setup and document ready

1. Include the official jQuery 4.0.0 CDN script in the HTML head, with a local fallback.
2. Load the external script.js after the library and catalogue data.
3. Wrap the implementation in $(document).ready() and log “jQuery is ready!” in the console.

Result. The store initializes with a $100 wallet, rendered rating controls and working navigation. The offline test also verifies that the local jQuery copy renders all 45 game-rating inputs.



Browser result: Setup and document ready.

script.js / lines 2-9

```
2   $(document).ready(function () {
3     'use strict';
4     console.log('jQuery is ready!');
5     const games = window.HUB_GAMES;
6     const byId = new Map(games.map(game => [game.id, game]));
7     const storageKey = 'eggs-game-hub-v6';
8     const money = cents => '$' + (cents / 100).toFixed(2);
9     const reducedMotion = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
```

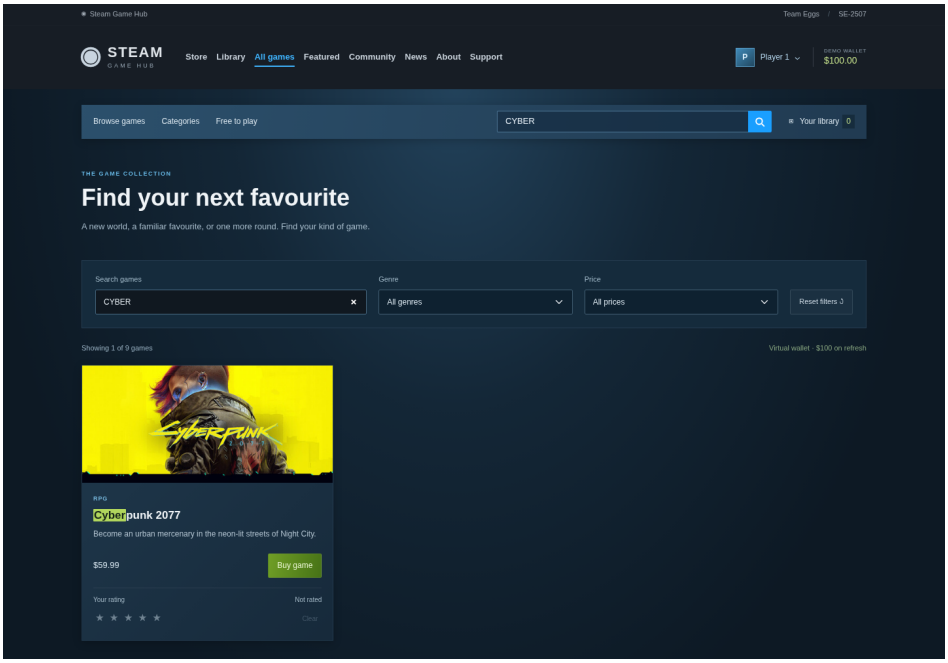
Source excerpt from the external script.js (line numbers shown).

TASK 1

Real-time game filtering

1. Bind input and change events to the search and filter fields.
2. Use jQuery .filter() to combine the search text, genre, price and library ownership.
3. Hide non-matching cards, update the count and display an empty state when needed.

Result. Typing CYBER leaves Cyberpunk 2077. RPG + Free to play produces an empty state. Reset filters restores all nine games. Searches ignore case and surrounding whitespace.



Browser result: Real-time game filtering.

script.js / lines 411-426

```
411      const $matches = $cards.filter(function () {
412        const game = byId.get($(this).attr('data-game'));
413        return (!isLibrary || profile().owned.includes(game.id)) &&
414          (genre === 'all' || genre === game.genre) && matchesPrice(game, budget) &&
415          (game.title + ' ' + game.genre).toLowerCase().includes(query.toLowerCase());
416      });
417      $cards.prop('hidden', true);
418      $matches.prop('hidden', false);
419      $cards.each(function () {
420        const game = byId.get($(this).attr('data-game'));
421        $(this).find('h3').html(highlighted(game.title, query));
422        $(this).find('.game-genre').html(highlighted(game.genre, query));
423        $(this).find('.game-description').html(highlighted(game.description, query));
424      });
425      const total = isLibrary ? profile().owned.length : games.length;
426      $('[data-catalogue-status]').text(`Showing ${$matches.length} of ${total} games`);
```

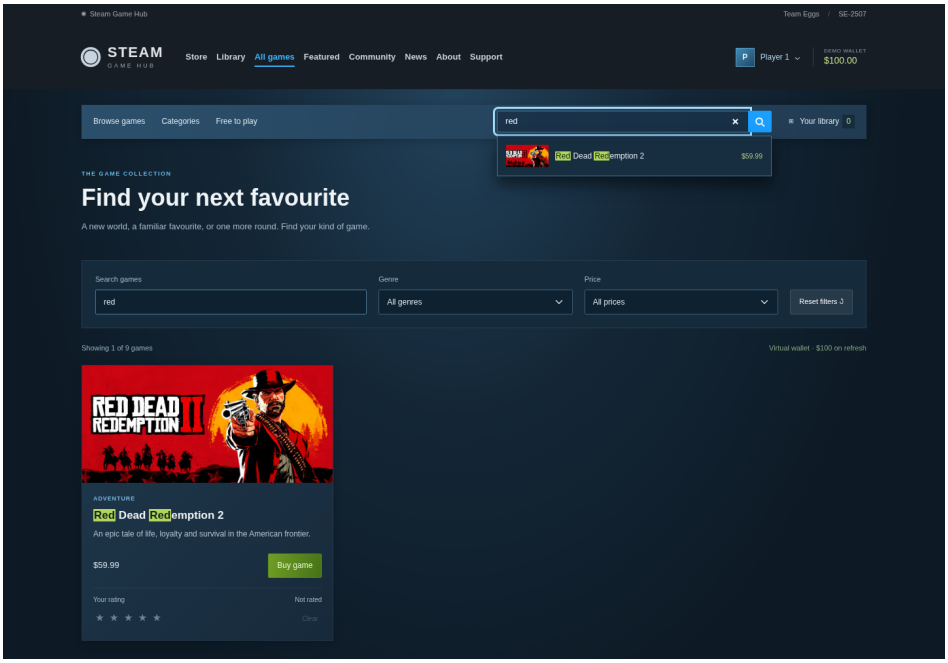
Source excerpt from the external script.js (line numbers shown).

TASK 2

Autocomplete suggestions

1. Read the search-bar value on input and find up to five matching games.
2. Build suggestions containing artwork, game name and price.
3. Select with a click, or use Up/Down and Enter. Escape and losing focus close the list.

Result. A suggestion opens the matching catalogue result. The combobox exposes aria-expanded, aria-controls and aria-activedescendant for keyboard and assistive-technology users.



Browser result: Autocomplete suggestions.

script.js / lines 444-463

```
444    function showSuggestions() {
445      const query = String($search.val()).trim();
446      $suggestions.empty();
447      selectedSuggestion = -1;
448      $search.removeAttr('aria-activedescendant');
449      if (!query) { closeSuggestions(); return; }
450      const matches = games.filter(g => (g.title + ' ' +
    g.genre).toLowerCase().includes(query.toLowerCase())).slice(0, 5);
451      matches.forEach((game, index) => {
452        const $option = $('<button type="button" role="option" aria-selected="false">').attr({ id:
    'suggestion-' + index, 'data-suggest': game.id, tabindex: '-1' });
453        $option.append($('<img alt="">').attr('src', game.image), $('<span>').html(highlighted(game.title,
     query)), $('<small>').text(game.cents ? money(game.cents) : 'Free'));
454        $suggestions.append($('<li role="presentation">').append($option));
455      });
456      if (!matches.length) $suggestions.append('<li class="suggestion-empty" role="presentation">No games
    found. Try another name or genre.</li>');
457      $suggestions.prop('hidden', false);
458      $search.attr('aria-expanded', 'true');
459    }
460    function chooseSuggestion(id) {
461      const game = byId.get(id);
462      if ($catalogue.length) {
463        $('#game-search, #store-search').val(game.title);
```

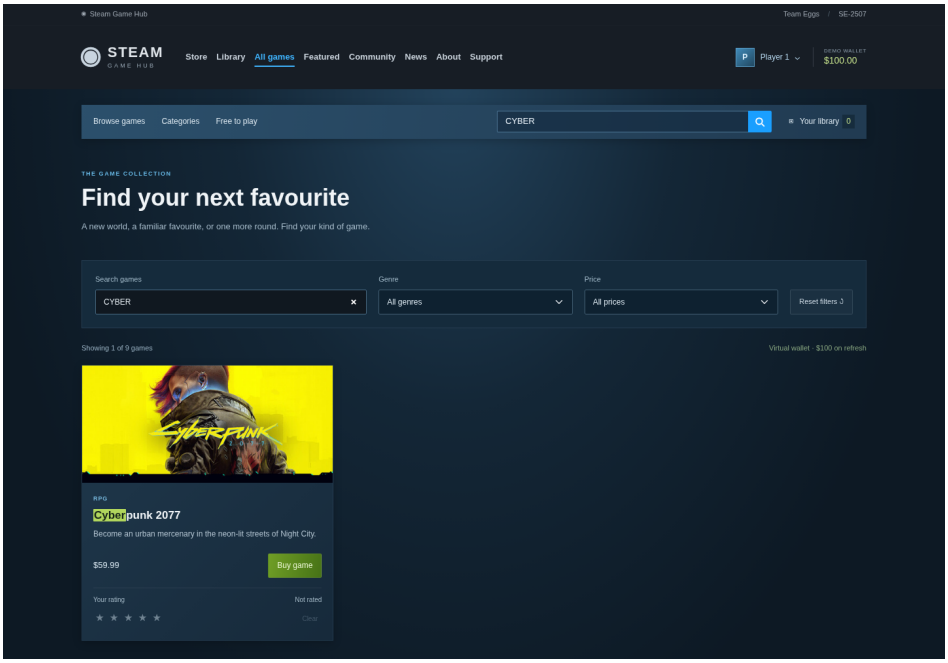
Source excerpt from the external script.js (line numbers shown).

TASK 3

Safe search highlighting

1. Escape regular-expression metacharacters in the search string.
2. Split each displayed title, genre and description around every matching text segment.
3. Escape the text as HTML, wrap matches in mark elements, and update the element through .html().

Result. Matching text is highlighted without changing the original game data. Inputs such as [, .* and HTML markup remain literal strings; clearing the query restores normal text.



Browser result: Safe search highlighting.

script.js / lines 388-396

```
388    function highlighted(text, query) {
389      if (!query) return escapeHTML(text);
390      // Escape regex metacharacters. User input is never inserted as raw HTML.
391      const pattern = query.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
392      const parts = text.split(new RegExp('(' + pattern + ')', 'gi'));
393      return parts.map((part, index) => index % 2 ? '<mark>' + escapeHTML(part) + '</mark>' :
     escapeHTML(part)).join('');
394    }
395    function matchesPrice(game, budget) {
396      switch (budget) {
```

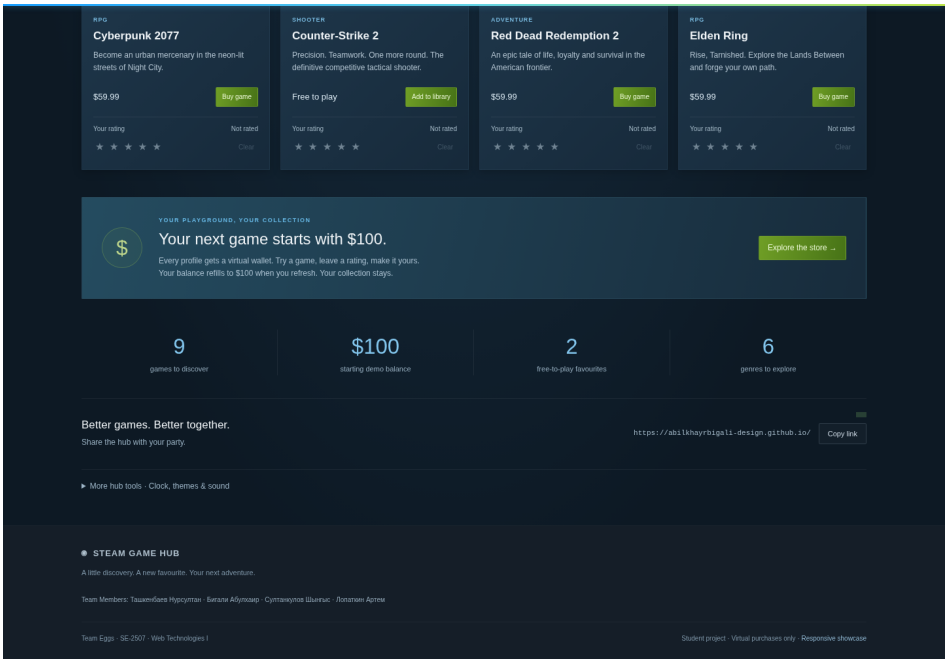
Source excerpt from the external script.js (line numbers shown).

TASK 4

Scroll progress indicator

1. Add a fixed three-pixel progress track at the top of the viewport.
2. On scroll and resize, divide scrollY by the available scroll distance.
3. Update the coloured bar width and clamp it to the valid range.

Result. The blue-to-green gradient fills as the user scrolls and reaches 100% at the bottom. Short pages handle zero scroll distance without division by zero.



Browser result: Scroll progress indicator.

script.js / lines 61-68

```
61    function updateScrollProgress() {
62      const max = document.documentElement.scrollHeight - window.innerHeight;
63      $('.scroll-meter > span').css('width', (max > 0 ? Math.min(100, window.scrollY / max * 100) : 0) +
    '%');
64    }
65    $(window).on('scroll resize', updateScrollProgress);
66    updateScrollProgress();
67
68    // Enhance the older Featured page cards with the same purchases and ratings.
```

Source excerpt from the external script.js (line numbers shown).

TASK 5

Animated number counters

1. Store each target number in a data-counter attribute.
2. Detect when the counter enters the viewport.
3. Animate an object property with jQuery .animate(), update the text on each step and set the exact final value.

Result. The home page counts to 9 games, $100 starting balance, 2 free games and 6 genres. Each counter runs once. Reduced-motion settings display the final number immediately.



Browser result: Animated number counters.

script.js / lines 508-523

```
508   function animateCounters() {
509     $('[data-counter]:not([data-counted])').each(function () {
510       if (this.getBoundingClientRect().top >= window.innerHeight) return;
511       const $element = $(this).attr('data-counted', 'true');
512       const target = Number($element.attr('data-counter'));
513       const prefix = $element.attr('data-counter-prefix') || '';
514       $({ count: 0 }).animate({ count: target }, {
515         duration: reducedMotion ? 0 : 1100,
516         step(value) { $element.text(prefix + Math.floor(value)); },
517         complete() { $element.text(prefix + target); },
518       });
519     });
520   }
521   $(window).on('scroll resize', animateCounters);
522   animateCounters();
523
```

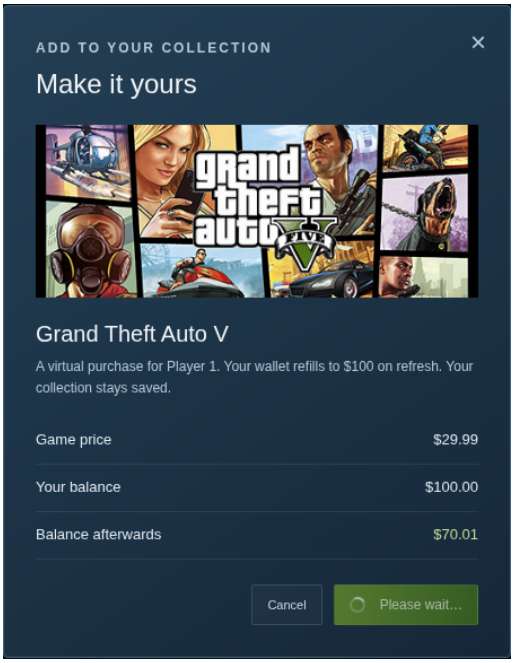
Source excerpt from the external script.js (line numbers shown).

TASK 6

Submit spinner and lock

1. Validate the purchase or return and show the amount before confirmation.
2. On submit, disable the button, display a spinner and “Please wait…” and guard against duplicate submissions.
3. After the simulated delay, re-check the current profile and balance, save the result and restore the controls.

Result. No actual server call occurs. The same feedback is added to the existing validated contact and subscription forms. Invalid forms show errors before any loading state.



Browser result: Submit spinner and lock.

script.js / lines 304-318

```
304    $('#checkout-form').on('submit', function (event) {
305      event.preventDefault();
306      if (busy || !transaction) return;
307      busy = true;
308      const pending = { ...transaction };
309      const $confirm = $('[data-checkout-confirm]');
310      $confirm.prop('disabled', true).html('<span class="spinner" aria-hidden="true"></span>Please
     wait…');
311      $checkout.attr('aria-busy', 'true').find('[data-close-hub-dialog]').prop('disabled', true);
312      setTimeout(function () {
313        state = readState();
314        const p = state.profiles.find(p => p.id === pending.profileId);
315        const game = byId.get(pending.id);
316        let error = '';
317        if (!p || state.active !== pending.profileId) error = 'The active player changed. Please try
     again.';
318        else if (pending.kind === 'buy') {
```

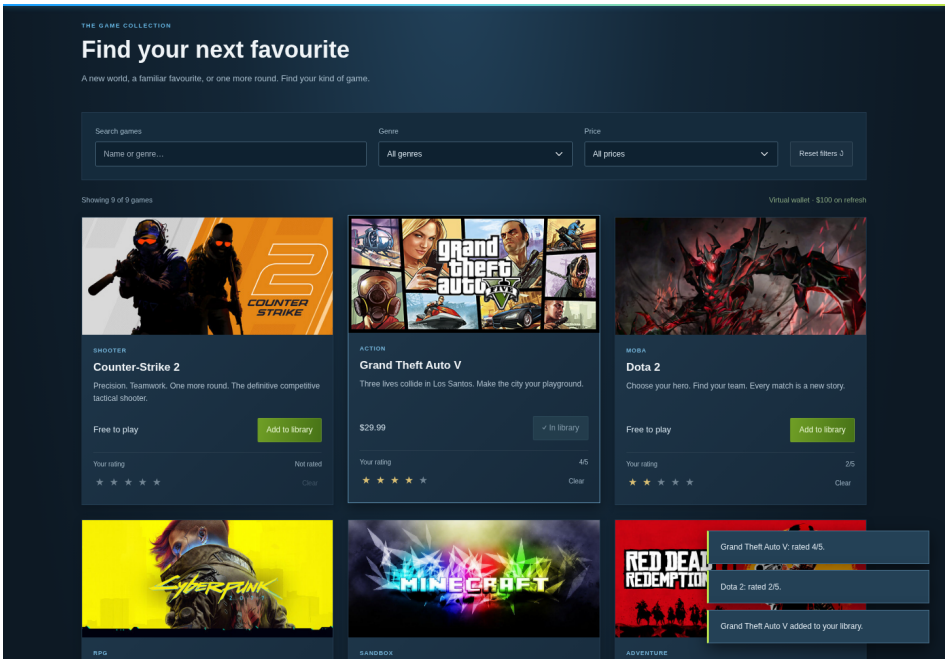
Source excerpt from the external script.js (line numbers shown).

TASK 7

Temporary notifications

1. Add an aria-live status area for notifications.
2. Insert the action result with .text(), then animate .fadeIn().
3. Wait with .delay(), run .fadeOut() and remove the finished toast.

Result. Notifications confirm purchases, refunds, ratings, profile changes, form submission and copying. Storage or transaction failures show an error notification instead of a false success.



Browser result: Temporary notifications.

script.js / lines 54-59

```
54    function toast(message, error = false) {
55      const $toast = $('<div class="hub-toast">').toggleClass('is-error', error).text(message).hide();
56      $('.hub-toasts').append($toast);
57      $toast.fadeIn(reducedMotion ? 0 : 180).delay(3600).fadeOut(reducedMotion ? 0 : 300, function () {
    $(this).remove(); });
58    }
59
```

Source excerpt from the external script.js (line numbers shown).

TASK 8

Copy to clipboard

1. Read the share link and use navigator.clipboard.writeText() after the user clicks.
2. Provide a textarea-based fallback for browsers without the Clipboard API.
3. Show a checkmark, a “Copied to clipboard!” status and tooltip, then restore the button.

Result. The browser test checks the actual clipboard contents as well as the visible confirmation. If copying fails, the page asks the user to select the link manually.



Browser result: Copy to clipboard.

script.js / lines 525-545

```
525    $('[data-copy-link]').on('click', async function () {
526      const $button = $(this);
527      const text = $('[data-share-url]').text();
528      try {
529        if (navigator.clipboard?.writeText) await navigator.clipboard.writeText(text);
530        else {
531          const $input = $('<textarea readonly>').val(text).css({ position: 'fixed', left: '-9999px'
    }).appendTo('body');
532          $input[0].select();
533          const copied = document.execCommand('copy');
534          $input.remove();
535          $button.trigger('focus');
536          if (!copied) throw new Error('Copy unavailable');
537        }
538        $button.text('□ Copied').attr('title', 'Copied to clipboard!');
539        $('[data-copy-status]').text('Copied to clipboard!');
540        toast('Hub link copied to clipboard.');
541        clearTimeout($button.data('copy-timeout'));
542        $button.data('copy-timeout', setTimeout(function () { $button.text('Copy
    link').removeAttr('title'); $('[data-copy-status]').empty(); }, 2500));
543      } catch { $('[data-copy-status]').text('Copy unavailable. Select the link to copy it manually.'); }
544    });
545
```

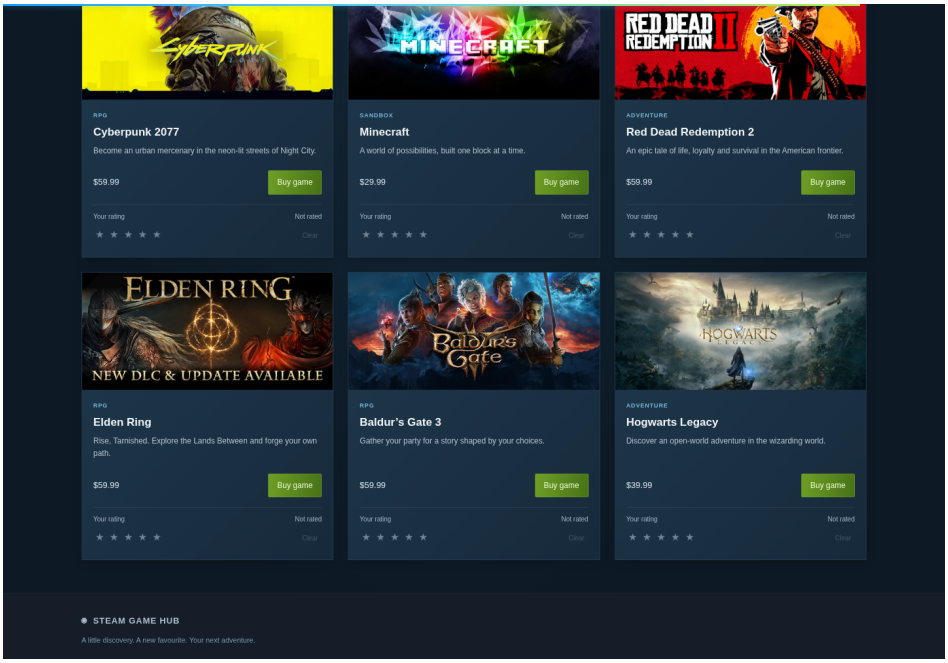
Source excerpt from the external script.js (line numbers shown).

TASK 9

Lazy image loading

1. Give off-screen images a small local SVG placeholder and store their real URL in data-src.
2. On scroll, resize and filter changes, check visible images with jQuery .offset().
3. Set .attr("src") only near the viewport, then remove data-src so the image is processed once.

Result. Lower catalogue images stay deferred until the user scrolls towards them. Images in hidden filter results or an empty library remain unloaded. Width and height keep card layout stable.



Browser result: Lazy image loading.

script.js / lines 498-508

```
498  function loadVisibleImages() {
499    const bottom = window.scrollY + window.innerHeight + 120;
500    $('img[data-src]').each(function () {
501      if (this.getClientRects().length && $(this).offset().top < bottom) {
502        $(this).attr('src', $(this).attr('data-src')).removeAttr('data-src');
503      }
504    });
505  }
506  $(window).on('scroll resize', loadVisibleImages);
507  // Task 5: animate counters once, when their section becomes visible.
508  function animateCounters() {
```

Source excerpt from the external script.js (line numbers shown).

ADDITIONAL REQUESTED FEATURES

Profiles, purchases and returns

Player profiles. Use the player name in the header to create or switch a local profile. Each profile starts with 10,000 integer cents ($100), an empty library and independent ratings. Duplicate and blank names are rejected.

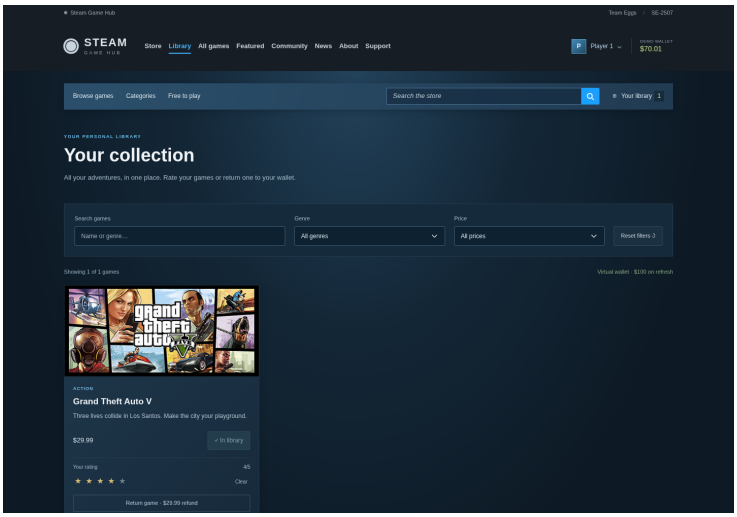
Purchases. A confirmation dialog shows the game price, current funds and remaining balance. A successful purchase subtracts the exact number of cents and adds the game ID once. Already-owned games cannot be bought again, and insufficient funds disable confirmation.

Refresh behaviour. As requested, an actual page reload refills every saved wallet to $100, detected using the navigation performance entry. Ordinary navigation preserves the current balance. Libraries and ratings survive refresh.

Returns. Library Return game removes ownership and adds the full virtual price. Returning a free game changes no funds. A purchase cannot be refunded twice. A refund after a refill may legitimately raise this demo wallet above $100.



Independent local profiles and their wallet balances.



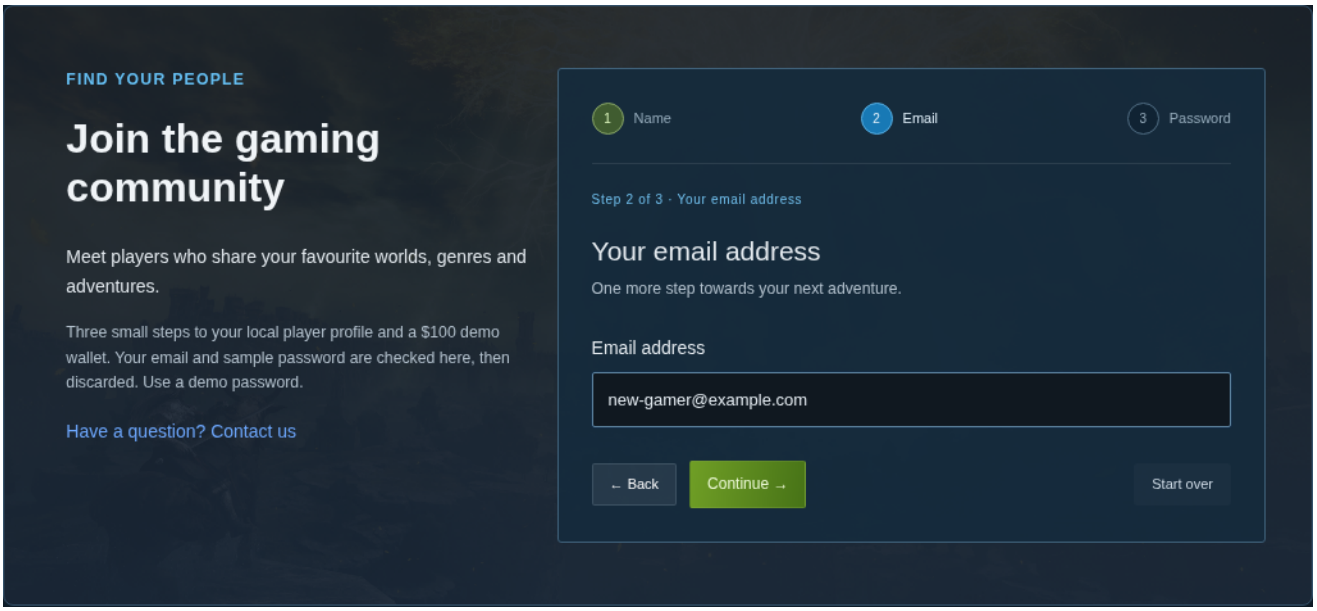
The library exposes a return action on each owned game.

COMMUNITY REGISTRATION

One block, three steps

1. Enter a unique player name and press Continue.
2. Enter a valid email address. Back returns to the previous field without clearing it.
3. Enter a demo password with at least eight characters and confirm it. Submit creates the local player profile and its $100 wallet.

Only the current step is visible. The step indicator, validation messages and focused input update in place. Empty fields, invalid email, short or mismatched passwords and existing player names prevent advancement. Start over clears the form.



Community registration, step 2 of 3. The same block changes from name to email to password.

script.js / lines 199-215

```
199     function showRegistrationStep(step, moveFocus = true) {
200       registrationStep = step;
201       $panels.each(function (index) { $(this).prop('hidden', index !== step); });
202       $registration.find('[data-registration-progress]').text(`Step ${step + 1} of 3 ·
    ${registrationLabels[step]}`);
203       $registration.find('[data-registration-indicator]').each(function (index) {
204         $(this).toggleClass('is-complete', index < step);
205         if (index === step) $(this).attr('aria-current', 'step');
206         else $(this).removeAttr('aria-current');
207       });
208       $registration.find('[data-registration-back]').prop('hidden', step === 0);
209       $registration.find('[data-registration-next]').text(step === 2 ? 'Create player profile' :
    'Continue →');
210       $registration.find('[data-registration-status]').empty();
211       if (moveFocus) $panels.eq(step).find('input').first().trigger('focus');
212     }
213     function resetRegistration() {
214       if (registrationBusy) return;
215       $registration[0].reset();
```

The step controller switches panels, progress, buttons and focus.

Completion. The submit spinner prevents duplicate profiles. A success panel confirms the active player and $100 wallet. Email and password are cleared and are never stored. Tests cover the whole flow at desktop and mobile widths.

VERIFICATION AND DEPLOYMENT

Checks, publication and reflection

Browser verification. 57/57 checks passed. The suite covers combined filters, literal regex/HTML inputs, keyboard suggestions, persistent per-game ratings, one-time charges, insufficient funds, returns, independent profiles, reload refills, clipboard contents, counters, progress, lazy images and form loading feedback.

Responsive checks. All ten pages were checked at 375, 768 and 1440 pixels. No horizontal page overflow, duplicate IDs, broken loaded images, JavaScript errors or backend submissions were detected in the checked scenarios. All local file and fragment links resolve.

How to reproduce. Run npm install, then npm test with Chromium available (BROWSER_PATH selects another Chromium executable). The site itself needs no build or Node.js. Open index.html, or serve the directory with python3 -m http.server 8765. For consistent cross-page storage, use HTTP or GitHub Pages.

Deployment. Changes were committed and pushed to the existing shared GitHub repository. GitHub Pages completed successfully. The deployed source was checked against local file hashes; details are in report/assignment6/deployment-verification.json.

Live website: https://abilkhayrbigali-design.github.io/

Challenges and solutions. Browser storage must keep each player independent, so purchases and ratings are keyed by stable profile and game IDs. Money uses integer cents to avoid floating-point rounding. A reload differs from navigation, so only the reload path refills wallets. Search highlighting escapes both regex and HTML. Transactions re-read saved state before completion to guard against stale data from another tab.

Reflection. jQuery simplifies selecting elements, reacting to input, updating DOM content and animating feedback. Browser-only state is enough for this academic demonstration; profiles are local data, not authenticated accounts. The same UI patterns can be explained and demonstrated without any backend.

Submission. The ZIP contains the complete frontend, local assets and libraries, this PDF, the test suite and evidence. Each team member must submit both PDF and ZIP in LMS by its stated deadline and defend the implementation during the practice lesson.

Files to review: script.js; js/games-data.js; css/store.css; tests/assignment6.cjs; report/assignment6/verification.json.